

HOJA DE RUTA OPERATIVA POR MÓDULOS

Nivelación y adaptación técnica para pasante de desarrollo de software

Documento complementario al plan modular principal

Campo	Detalle
Documento	Hoja de ruta operativa por módulos
Dirigido a	Pasante de desarrollo de software
Complementa a	Plan modular de nivelación y adaptación técnica
Propósito	Guiar la ejecución práctica de cada módulo mediante inputs, temas, ejercicios, entregables y criterios de aceptación.
Enfoque	Acompañamiento formativo, comunicación profesional, aprendizaje progresivo y uso responsable de IA.

Mensaje central

Este documento está diseñado para acompañar tu proceso de aprendizaje de manera clara, progresiva y profesional. No se espera que domines todos los temas desde el inicio. Sí se espera que puedas comunicar avances, dudas, límites, bloqueos y aprendizajes con honestidad y orden.

1. Propósito del documento

Esta hoja de ruta complementa el plan modular de nivelación y adaptación técnica. Mientras el plan principal define los objetivos generales, este documento aterriza la ejecución práctica de cada módulo: qué revisar, qué investigar, qué ejercicios realizar, qué evidencias entregar y cómo preparar las revisiones.

El enfoque es formativo. La intención no es generar presión innecesaria, sino dar claridad sobre el camino de aprendizaje y permitir que el avance sea observable mediante evidencias concretas.

- Guiar el aprendizaje paso a paso sin convertirlo en una evaluación rígida.
- Promover comunicación profesional, especialmente frente a dudas o bloqueos.
- Fortalecer fundamentos técnicos antes de asumir tareas de mayor responsabilidad.
- Asegurar que el uso de IA sea un apoyo y no un reemplazo del razonamiento propio.
- Facilitar conversaciones de seguimiento basadas en evidencias, no en percepciones aisladas.

2. Cómo utilizar esta hoja de ruta

Cada módulo debe trabajarse con una dinámica sencilla: revisar el objetivo, estudiar los temas mínimos, realizar los ejercicios, preparar entregables y completar la bitácora. Cuando exista evidencia suficiente, se solicita revisión.

Paso	Acción esperada
1	Leer el objetivo del módulo y confirmar qué se espera lograr.
2	Investigar los temas indicados con fuentes, práctica y ejemplos propios.
3	Realizar los ejercicios prácticos y registrar errores o dudas.
4	Completar la bitácora con avances, bloqueos y aprendizajes reales.
5	Preparar los entregables y evidencias para revisión.
6	Solicitar retroalimentación con preguntas concretas.
7	Aplicar ajustes y registrar qué cambió después de la revisión.
Criterio de trabajo Avanzar no significa no tener dudas. Avanzar significa demostrar que las dudas se identifican, se investigan, se comunican y se trabajan con orden.	

3. Estructura estándar de cada módulo

Sección	Descripción
Objetivo del módulo	Define qué competencia se busca fortalecer o validar.
Input del módulo	Material, contexto o entregables previos necesarios para iniciar.
Temas a investigar	Conceptos mínimos que deben estudiarse antes y

	durante la práctica.
Ejercicios prácticos	Actividades concretas para aplicar los conceptos.
Uso permitido de IA	Reglas para usar IA de forma responsable y verificable.
Entregables esperados	Evidencias que deben presentarse al finalizar el módulo.
Ejemplo de bitácora	Modelo de registro esperado del aprendizaje.
Criterios de aceptación	Condiciones mínimas para considerar el módulo suficientemente trabajado.
Señales de refuerzo	Indicadores de que conviene repetir, ajustar o reforzar el módulo.

4. Hoja de ruta por módulos

Los módulos están organizados de menor a mayor complejidad. El foco inicial está en comunicación, fundamentos y razonamiento; luego se avanza hacia diseño, arquitectura, API, serverless e integración final.

Módulo 0. Acuerdo de trabajo, comunicación profesional y diagnóstico inicial

Objetivo del módulo

Conocer el punto de partida, alinear expectativas y establecer una forma de comunicación clara, responsable y profesional durante el proceso de nivelación.

Input del módulo

- Plan modular principal.
- Matriz de evaluación formativa.
- Contexto general del rol dentro del equipo.
- Lineamientos sobre comunicación de avances, dudas y bloqueos.
- Reglas de uso responsable de IA.

Temas a investigar

- Qué significa trabajar dentro de un proyecto de software.
- Diferencia entre no saber y no comunicar.
- Cómo pedir ayuda con contexto.
- Cómo reportar bloqueos técnicos.
- Qué es una bitácora de aprendizaje.
- Buenas prácticas de comunicación profesional.
- Cómo reconocer límites técnicos sin perder iniciativa.

Ejercicios prácticos

1. Elaborar un autodiagnóstico técnico inicial.
2. Identificar fortalezas actuales.
3. Identificar temas que necesitan refuerzo.
4. Redactar tres ejemplos de comunicación de bloqueos.
5. Redactar tres preguntas técnicas bien formuladas.

6. Preparar una conversación inicial de expectativas y dudas.

Uso permitido de IA

- Puede usarse IA para ordenar ideas, proponer formatos de bitácora o mejorar redacción.
- No se debe usar IA para aparentar dominio de temas no comprendidos.
- Todo texto generado con IA debe revisarse y adaptarse con palabras propias.

Entregables esperados

- Autodiagnóstico inicial.
- Lista de fortalezas.
- Lista de temas por reforzar.
- Ejemplos de comunicación de bloqueos.
- Primera bitácora breve.
- Preguntas iniciales para revisión.

Ejemplo de comunicación de bloqueo

La comunicación esperada debe permitir que otra persona entienda el contexto sin tener que adivinar.

Elemento	Ejemplo
Qué intento hacer	Estoy intentando ejecutar el proyecto de tareas desde consola.
Qué esperaba	Esperaba que el script mostrara el menú inicial.
Qué ocurrió	Aparece un error de importación al ejecutar main.py.
Qué intenté	Revisé el nombre de los archivos, ejecuté desde la raíz del proyecto y validé la versión de Python.
Pregunta concreta	¿La estructura de carpetas puede estar afectando el import?

Ejemplo de bitácora del módulo

Campo	Ejemplo de respuesta esperada
Qué aprendí	Aprendí que comunicar un bloqueo no significa fallar; significa dar visibilidad para avanzar mejor.
Qué practiqué	Redacté ejemplos de cómo pedir ayuda cuando no entiendo un error o una instrucción.
Dónde tuve dificultad	Me costó diferenciar entre una duda general y una pregunta concreta.
Qué hice para resolverlo	Reformulé mis preguntas agregando contexto, error observado y acciones intentadas.
Qué necesito reforzar	Necesito practicar cómo explicar problemas técnicos de forma más ordenada.

Criterios de aceptación

- Reconoce su nivel actual sin exagerar ni minimizar.
- Comunica dudas con claridad.
- Identifica temas que necesita reforzar.

- Muestra disposición para recibir retroalimentación.
- Entiende que la comunicación profesional forma parte del proceso de evaluación.

Señales de refuerzo

- Responde de forma muy general sin evidencias.
- Evita mencionar dudas o bloqueos reales.
- No logra formular preguntas concretas.
- Depende de frases generadas por IA sin adaptarlas a su contexto.

Módulo 1. Fundamentos prácticos de Python

Objetivo del módulo

Validar y fortalecer bases reales de programación mediante ejercicios simples, explicables y modificables en Python.

Input del módulo

- Editor de código configurado.
- Python instalado.
- Carpeta o repositorio de trabajo.
- Reglas básicas de entrega.
- Ejercicio base: gestor simple de tareas en memoria.

Temas a investigar

- Variables y tipos de datos.
- Condicionales.
- Bucles.
- Funciones.
- Listas.
- Diccionarios.
- Manejo básico de errores.
- Clases simples.
- Lectura de errores en consola.
- Ejecución de scripts Python.
- Diferencia entre escribir código y entender código.

Ejercicios prácticos

7. Crear una lista de tareas en memoria.
8. Agregar una tarea.
9. Listar tareas.
10. Marcar una tarea como completada.
11. Validar que no se creen tareas vacías.
12. Mostrar mensajes claros al usuario.
13. Separar el código en funciones.
14. Explicar qué hace cada función.
15. Modificar una validación durante la revisión.

Uso permitido de IA

- Puede usarse IA para consultar sintaxis, obtener ejemplos o pedir explicación de errores.
- Cada fragmento utilizado debe ser entendido y explicado.
- Debe registrar los prompts relevantes y qué modificó por cuenta propia.
- Durante la revisión, debe poder realizar un cambio pequeño sin depender completamente de IA.

Entregables esperados

- Archivo Python funcional.
- README breve con instrucciones de ejecución.

- Evidencia de ejecución.
- Bitácora del módulo.
- Lista de dudas encontradas.
- Registro de prompts usados, si aplicó IA.

Ejemplo mínimo esperado

Este ejemplo no representa la solución completa. Sirve como referencia de nivel inicial para validar estructura, funciones y lógica simple.

```
tareas = []

def agregar_tarea(nombre):
    if not nombre.strip():
        return "La tarea no puede estar vacía"

    tarea = {
        "nombre": nombre,
        "completada": False
    }

    tareas.append(tarea)
    return "Tarea agregada correctamente"

def listar_tareas():
    return tareas
```

Ejemplo de bitácora del módulo

Campo	Ejemplo de respuesta esperada
Qué aprendí	Aprendí a usar listas y diccionarios para guardar información sencilla.
Qué practiqué	Construí funciones para agregar, listar y completar tareas.
Dónde tuve dificultad	Me confundí al recorrer la lista y mostrar el estado de cada tarea.
Qué hice para resolverlo	Imprimí la estructura completa y luego accedí campo por campo.
Qué necesito reforzar	Necesito practicar más funciones, validaciones y estructuras de datos.

Criterios de aceptación

- Ejecuta el script sin errores principales.
- Explica la lógica general.
- Puede modificar una validación simple.
- Entiende funciones, listas y diccionarios.
- No depende totalmente de IA para cambios menores.
- Entrega instrucciones mínimas para ejecutar el ejercicio.

Señales de refuerzo

- No puede ejecutar su propio código.
- No puede explicar qué hace una función.
- Copia código sin comprenderlo.
- No registra errores ni aprendizajes.
- No puede realizar cambios simples guiados.

Módulo 2. Programación orientada a objetos y SOLID básico

Objetivo del módulo

Introducir separación de responsabilidades, organización del código y principios básicos de diseño mantenible.

Input del módulo

- Código del módulo anterior.
- Ejemplo de código mezclado o poco organizado.
- Lectura introductoria sobre POO y SOLID.
- Ejercicio de refactorización.
- Retroalimentación recibida en el módulo 1.

Temas a investigar

- Qué es una clase.
- Qué es un objeto.
- Qué es un método.
- Qué es una responsabilidad.
- Diferencia entre función y método.
- SRP: principio de responsabilidad única.
- OCP: abierto/cerrado.
- DIP: inversión de dependencias.
- Acoplamiento.
- Cohesión.
- Separación entre lógica de negocio y entrada/salida.

Ejercicios prácticos

16. Convertir el gestor de tareas a clases simples.
17. Crear una clase Task.
18. Crear una clase TaskService.
19. Separar almacenamiento en una clase distinta.
20. Identificar la responsabilidad de cada clase.
21. Escribir una explicación corta de SRP aplicada al ejercicio.
22. Mostrar un antes y después del código.
23. Responder qué cambiaría si el almacenamiento dejara de ser en memoria.

Uso permitido de IA

- Puede usarse IA para comparar ejemplos de diseño, pero debe justificar sus decisiones.
- No debe memorizar definiciones sin poder relacionarlas con su código.
- Debe registrar qué sugerencias de IA aceptó, rechazó o modificó.

Entregables esperados

- Código refactorizado.
- Documento corto explicando las clases creadas.
- Tabla antes vs después.
- Bitácora del módulo.

- Registro de prompts usados.
- Dudas pendientes sobre SOLID.

Ejemplo de análisis antes vs después

Antes	Después
Una sola función hacía varias cosas.	Se separó creación, almacenamiento y consulta.
La lógica estaba mezclada con impresión en consola.	La lógica principal se movió a un servicio.
Era difícil modificar reglas.	Las validaciones se ubicaron en un lugar más claro.

Ejemplo de bitácora del módulo

Campo	Ejemplo de respuesta esperada
Qué aprendí	Aprendí que SOLID no es memorizar letras, sino organizar el código para que sea más fácil cambiarlo.
Qué practiqué	Separé la creación de tareas de la lógica para almacenarlas.
Dónde tuve dificultad	Me costó entender cuándo una clase tiene demasiadas responsabilidades.
Qué hice para resolverlo	Comparé mi código antes y después del refactor.
Qué necesito reforzar	Necesito practicar DIP con ejemplos más simples.

Criterios de aceptación

- Identifica responsabilidades básicas.
- Explica SRP con su propio código.
- Puede justificar por qué separó una clase.
- Reconoce que no domina SOLID por completo, pero puede aplicarlo de forma inicial.
- No usa términos técnicos sin poder relacionarlos con el ejercicio.

Señales de refuerzo

- Explica SOLID solo como definiciones memorizadas.
- No puede identificar responsabilidades.
- Crea clases sin propósito claro.
- No puede explicar el cambio entre antes y después.
- Confunde arquitectura con cantidad de archivos.

Módulo 3. Clean Architecture introductoria

Objetivo del módulo

Comprender la separación entre reglas de negocio, casos de uso, infraestructura y mecanismos externos.

Input del módulo

- Código refactorizado del módulo anterior.
- Diagrama simple de capas.
- Ejemplo de estructura de carpetas.
- Explicación inicial de Clean Architecture.
- Preguntas pendientes de POO y SOLID.

Temas a investigar

- Qué es una arquitectura de software.
- Qué problema intenta resolver Clean Architecture.
- Dominio.
- Caso de uso.
- Infraestructura.
- Adaptadores.
- Entidad.
- Interfaz o contrato.
- Independencia del framework.
- Lógica de negocio vs detalle técnico.
- Regla de dependencia en Clean Architecture.

Ejercicios prácticos

24. Dibujar las capas de la aplicación de tareas.
25. Reorganizar carpetas en domain, use_cases, infrastructure e interfaces.
26. Mover la entidad Task al dominio.
27. Crear un caso de uso para crear tareas.
28. Crear un repositorio en memoria.
29. Explicar qué parte sería reemplazable en el futuro.
30. Identificar qué no debería depender de FastAPI o AWS.
31. Explicar el flujo de una operación simple.

Uso permitido de IA

- Puede usarse IA para obtener ejemplos de estructuras, pero la estructura final debe explicarse con el ejercicio propio.
- Debe evitar generar una arquitectura excesiva para un problema pequeño.
- Debe documentar dudas conceptuales, especialmente dominio, caso de uso e infraestructura.

Entregables esperados

- Diagrama simple de capas.
- Estructura de carpetas.
- Código reorganizado.

- Explicación breve de cada capa.
- Bitácora.
- Lista de dudas conceptuales.

Ejemplo de estructura esperada

```
task_app/
  domain/
    task.py
  use_cases/
    create_task.py
    list_tasks.py
  infrastructure/
    in_memory_task_repository.py
  interfaces/
    cli.py
```

Ejemplo de bitácora del módulo

Campo	Ejemplo de respuesta esperada
Qué aprendí	Aprendí que Clean Architecture busca proteger la lógica de negocio de detalles externos.
Qué practiqué	Reorganicé el proyecto en carpetas por responsabilidad.
Dónde tuve dificultad	Me costó decidir qué va en dominio y qué va en infraestructura.
Qué hice para resolverlo	Revisé cada archivo y pregunté si representaba negocio o detalle técnico.
Qué necesito reforzar	Necesito practicar casos de uso y repositorios.

Criterios de aceptación

- Explica la diferencia entre dominio e infraestructura.
- Ubica una regla de negocio en la capa correcta.
- Entiende que una estructura de carpetas no garantiza arquitectura.
- Explica el flujo de una operación simple.
- Reconoce qué partes podrían cambiar sin afectar el negocio.

Señales de refuerzo

- Solo mueve archivos sin entender responsabilidades.
- Confunde framework con arquitectura.
- No puede explicar por qué una capa depende o no de otra.
- Sobrediseña el ejercicio sin necesidad.
- No distingue regla de negocio de detalle técnico.

Módulo 4. API básica y calidad mínima de entrega

Objetivo del módulo

Construir una API sencilla y comprensible, aplicando orden, validaciones básicas y documentación mínima.

Input del módulo

- Código del módulo anterior.
- FastAPI instalado.
- Ejemplo básico de endpoint.
- Reglas mínimas de documentación.
- Herramienta de prueba: navegador, curl, Postman o Swagger.

Temas a investigar

- Qué es una API REST.
- Qué es un endpoint.
- Métodos HTTP: GET, POST, PUT, PATCH y DELETE.
- Códigos de estado básicos.
- Request y response.
- JSON.
- Validación de datos.
- Manejo básico de errores.
- Swagger/OpenAPI.
- README técnico.
- Pruebas manuales de endpoints.

Ejercicios prácticos

32. Crear endpoint para registrar tarea.
33. Crear endpoint para listar tareas.
34. Crear endpoint para marcar tarea como completada.
35. Validar que el nombre no esté vacío.
36. Retornar errores claros.
37. Probar los endpoints.
38. Documentar cómo ejecutar el proyecto.
39. Explicar el flujo request - caso de uso - repositorio - response.

Uso permitido de IA

- Puede usarse IA para consultar sintaxis de FastAPI, ejemplos de modelos o códigos HTTP.
- Debe validar manualmente los endpoints y registrar evidencias.
- Debe explicar qué parte pertenece a la API y qué parte pertenece al caso de uso.

Entregables esperados

- API funcional.
- README con pasos de ejecución.
- Evidencia de pruebas.
- Tabla de endpoints.

- Bitácora.
- Registro de errores encontrados y cómo fueron resueltos.

Ejemplo de tabla de endpoints

Método	Endpoint	Descripción
POST	/tasks	Crea una tarea.
GET	/tasks	Lista tareas.
PATCH	/tasks/{id}/complete	Marca una tarea como completada.

Ejemplo de bitácora del módulo

Campo	Ejemplo de respuesta esperada
Qué aprendí	Aprendí cómo una API recibe datos y responde en formato JSON.
Qué practiqué	Creé endpoints para crear y listar tareas.
Dónde tuve dificultad	Me confundí con los códigos de estado HTTP.
Qué hice para resolverlo	Probé respuestas y revisé la documentación de FastAPI.
Qué necesito reforzar	Necesito practicar validaciones y manejo de errores.

Criterios de aceptación

- La API ejecuta correctamente.
- Los endpoints principales responden.
- El README permite que otra persona ejecute el proyecto.
- Puede explicar el flujo general.
- Puede corregir una observación concreta de forma guiada.
- Incluye evidencias de prueba.

Señales de refuerzo

- La API solo funciona en su equipo por configuraciones no documentadas.
- No puede explicar request/response.
- No valida entradas básicas.
- No registra pruebas.
- No diferencia endpoint de lógica de negocio.

Módulo 5. AWS SAM y nociones de despliegue serverless

Objetivo del módulo

Comprender el propósito de AWS SAM, la estructura básica de un template y la relación entre código, infraestructura y despliegue.

Input del módulo

- Ejemplo de proyecto AWS SAM.
- Archivo template.yaml.
- Explicación básica de Lambda y API Gateway.
- Código simple de una Lambda.
- Documentación base de comandos SAM.
- Contexto de despliegue usado por el equipo, si aplica.

Temas a investigar

- Qué es serverless.
- Qué es AWS Lambda.
- Qué es API Gateway.
- Qué es AWS SAM.
- Qué es un template YAML.
- Infraestructura como código.
- Diferencia entre código de aplicación e infraestructura.
- sam build.
- sam local start-api.
- sam deploy.
- Variables de entorno.
- Logs básicos en CloudWatch.
- Buenas prácticas mínimas de despliegue.

Ejercicios prácticos

40. Revisar un template SAM existente.
41. Identificar recursos principales.
42. Explicar qué recurso representa la Lambda.
43. Explicar qué recurso representa el API.
44. Ejecutar localmente si el ambiente lo permite.
45. Modificar una variable de entorno.
46. Documentar el flujo de despliegue.
47. Proponer una mejora simple al template.
48. Dibujar el flujo API Gateway - Lambda - respuesta.

Uso permitido de IA

- Puede usarse IA para explicar secciones del template, pero debe contrastar con el archivo real.
- Debe registrar qué recursos entendió y cuáles necesita reforzar.
- No debe ejecutar despliegues reales sin autorización o guía del equipo.

Entregables esperados

- Análisis del template.yaml.
- Tabla de recursos identificados.
- Diagrama simple del flujo API Gateway - Lambda.
- README con comandos básicos.
- Bitácora.
- Lista de dudas sobre AWS/SAM.

Ejemplo de tabla de recursos SAM

Recurso	Tipo	Para qué sirve
TaskFunction	AWS::Serverless::Function	Ejecuta la lógica de la Lambda.
TaskApi	AWS::Serverless::Api	Expone endpoints HTTP.
Environment	Variables	Configura valores usados por la función.

Ejemplo de bitácora del módulo

Campo	Ejemplo de respuesta esperada
Qué aprendí	Aprendí que SAM permite definir infraestructura serverless en un archivo YAML.
Qué practiqué	Revisé un template e identifiqué Lambda y API Gateway.
Dónde tuve dificultad	Me costó entender la diferencia entre build y deploy.
Qué hice para resolverlo	Leí los comandos y armé una explicación paso a paso.
Qué necesito reforzar	Necesito practicar más lectura de templates YAML.

Criterios de aceptación

- Distingue código de aplicación e infraestructura.
- Identifica recursos principales en un template.
- Explica el flujo básico de una petición.
- Comprende que SAM no es solo ejecutar comandos.
- Documenta el proceso con orden.
- Respeta límites de ejecución y despliegue.

Señales de refuerzo

- Confunde Lambda con API Gateway.
- No puede leer recursos básicos del template.
- Ejecuta comandos sin comprender impacto.
- No diferencia ambiente local de despliegue real.
- No registra dudas ni riesgos.

Módulo 6. Integración final, comunicación técnica y retrospectiva

Objetivo del módulo

Integrar lo aprendido en una entrega pequeña, explicable y defendible, reforzando comunicación, criterio técnico y autoconocimiento profesional.

Input del módulo

- Entregables de módulos anteriores.
- Bitácoras acumuladas.
- Feedback recibido.
- Matriz de evaluación.
- Proyecto final pequeño.
- Lista de dudas abiertas.

Temas a investigar

- Cómo preparar una demo técnica.
- Cómo explicar decisiones técnicas.
- Cómo presentar limitaciones.
- Cómo documentar próximos pasos.
- Cómo recibir feedback.
- Cómo hacer una retrospectiva personal.
- Diferencia entre entregar código y comunicar una solución.
- Cómo evidenciar evolución profesional.

Ejercicios prácticos

49. Preparar una demo breve del ejercicio.
50. Explicar el flujo completo de la solución.
51. Mostrar una mejora aplicada a partir de feedback.
52. Explicar qué parte entiende mejor.
53. Explicar qué parte necesita reforzar.
54. Presentar próximos pasos personales.
55. Responder preguntas técnicas sobre su propio código.
56. Proponer tareas donde podría apoyar al equipo.

Uso permitido de IA

- Puede usarse IA para preparar la estructura de la presentación, no para reemplazar la comprensión.
- Debe poder responder preguntas sobre su propio código sin leer una respuesta generada.
- Debe declarar con claridad qué partes fueron apoyadas por IA y cómo las validó.

Entregables esperados

- Demo breve.
- Repositorio o carpeta final organizada.
- README final.
- Bitácora consolidada.
- Retrospectiva personal.
- Lista de próximos temas de refuerzo.

- Registro de uso de IA durante el proceso.

Ejemplo de retrospectiva final

Pregunta	Respuesta esperada
Qué aprendí mejor	Aprendí mejor la separación de responsabilidades y la creación de endpoints simples.
Qué me costó más	Me costó entender Clean Architecture y diferenciar dominio de infraestructura.
Qué haría diferente	Dividiría los cambios en pasos más pequeños y documentaría errores desde el inicio.
Dónde necesito apoyo	Necesito reforzar POO, pruebas y lectura de errores.
En qué puedo aportar ahora	Puedo apoyar en documentación, pruebas manuales, scripts simples y lectura guiada de código.

Ejemplo de bitácora del módulo

Campo	Ejemplo de respuesta esperada
Qué aprendí mejor	Aprendí mejor la separación básica de responsabilidades.
Qué me costó más	Me costó entender Clean Architecture y el rol de infraestructura.
Qué haría diferente	Haría cambios más pequeños y documentaría dudas antes.
Dónde necesito apoyo	Necesito reforzar POO, pruebas y lectura de errores.
En qué puedo aportar ahora	Puedo apoyar en documentación, pruebas manuales, scripts simples y lectura guiada de código.

Criterios de aceptación

- Presenta con orden.
- Explica su propio código.
- Reconoce límites sin incomodidad.
- Muestra evolución frente al diagnóstico inicial.
- Identifica próximos pasos realistas.
- Demuestra mejor comunicación técnica.
- Recibe feedback sin bloquearse.

Señales de refuerzo

- No puede explicar decisiones tomadas.
- Evita hablar de dificultades reales.
- No muestra evolución respecto al diagnóstico inicial.
- No conecta entregables con aprendizajes.
- Depende de respuestas memorizadas o generadas.

5. Plantillas anexas

Estas plantillas permiten estandarizar la documentación del aprendizaje. Pueden copiarse por cada módulo y actualizarse conforme avance el proceso.

5.1 Plantilla de bitácora por módulo

Campo	Respuesta
Módulo trabajado	
Fecha de registro	
Qué investigué	
Qué entendí	
Qué practiqué	
Qué entregué	
Qué errores tuve	
Cómo intenté resolverlos	
Qué prompts usé	
Qué validé por mi cuenta	
Qué necesito reforzar	
Preguntas para revisión	

5.2 Plantilla de registro de uso de IA

Campo	Respuesta
Herramienta utilizada	
Prompt usado	
Respuesta obtenida	
Qué parte entendí	
Qué parte modifiqué	
Qué validé manualmente	
Qué duda quedó pendiente	

5.3 Plantilla para reportar bloqueos

Campo	Respuesta
Qué intento hacer	
Qué esperaba que ocurra	
Qué ocurrió realmente	
Error o comportamiento observado	
Qué intenté para resolverlo	
Qué investigué	
Pregunta concreta	

Ejemplo de reporte de bloqueo

Estoy intentando ejecutar la API con FastAPI. Esperaba que Swagger cargue en /docs, pero recibo un error de importación. Ya revisé el nombre del archivo, reinstalé dependencias y validé el entorno virtual. Mi duda concreta es si la estructura de carpetas está afectando el import.

5.4 Plantilla de revisión por módulo

Criterio	Observación del pasante	Observación del líder
Comprensión técnica		
Comunicación		
Uso de IA		
Calidad del entregable		
Autonomía		
Próximo refuerzo		

5.5 Plantilla de checklist de entrega

Elemento	Cumple	Observación
Código o evidencia principal entregada		
README o instrucciones incluidas		
Bitácora completada		
Dudas registradas		
Prompts de IA registrados, si aplican		
Evidencia de pruebas incluida		
Puede explicar lo entregado		

6. Criterios generales de salida

El proceso habrá cumplido su propósito cuando el pasante demuestre avance visible en fundamentos, comunicación y criterio profesional. La salida no implica dominio total de todos los temas, sino una base más clara para definir próximos pasos y asignaciones adecuadas.

- Puede explicar lo que hizo y por qué lo hizo.
- Comunica límites, dudas y bloqueos con mayor claridad.
- Utiliza IA como apoyo, no como sustituto completo del razonamiento.
- Entrega evidencias simples pero ordenadas.
- Muestra comprensión inicial de Python, POO, SOLID, Clean Architecture, API y AWS SAM.
- Recibe feedback y aplica mejoras concretas.
- Identifica próximos temas de refuerzo con honestidad.
- Puede integrarse gradualmente a tareas acordes con su nivel actual.

Cierre formativo

El resultado esperado no es perfección técnica inmediata. El resultado esperado es evolución: más claridad, mejores preguntas, entregas más ordenadas y una forma de trabajo que permita acompañamiento efectivo dentro del equipo.